



St. Francis Institute of Technology
(An Autonomous Institution)
AICTE Approved | Affiliated to University of Mumbai
A+ Grade by NAAC: CMPN, EXTC, INFT NBA Accredited: ISO 9001:2015 Certified

Department of Information Technology

MINI-PROJECT REPORT

AI&ML in Healthcare

Mini-Project (HAIMLSBL701)

VisionGuard – Intelligent Eye Disease Detection System

B. E. Information Technology

By

Sachi Jadhav	7
Tanmay Bhatkar	8
Shruti Kadam	9
Durva Kadam	11

Guide

Ms. Divyashree Karangutkar
Assistant Professor

Academic Year: 2025-2026

CERTIFICATE

This **AI&ML in Healthcare Mini-Project ‘VisionGuard – Intelligent Eye Disease Detection System’** by **‘Sachi Jadhav- 7, Tanmay Bhatkar- 8, Shruti Kadam-9, Durva Kadam-11’** is complete in all respects and was successfully demonstrated on _____.

Ms. Divyashree Karangutkar
(Mini Project Guide)

Dr. Prachi Raut
(HOD-INFT)

Examiners

1.-----

2.-----

Date:
Place:

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources.

We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in this submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

1. -----
Sachi Jadhav - 7

2. -----
Tanmay Bhatkar - 8

3. -----
Shruti Kadam - 9

4. -----
Durva Kadam - 11

Date:

ABSTRACT

VisionGuard, Intelligent Eye Disease Detection System is an advanced deep learning-based solution designed to assist in the early and accurate diagnosis of common eye disorders using retinal images. Visual impairments caused by diseases such as Cataract, Glaucoma, and Diabetic Retinopathy often progress silently and remain undetected until significant vision loss occurs. Traditional diagnostic methods require expert ophthalmologists and are time-consuming, especially when screening large populations. To address these challenges, VisionGuard leverages the power of Convolutional Neural Networks (CNNs) to automatically classify retinal fundus images into distinct categories, thereby improving diagnostic efficiency and reliability.

The system begins with dataset acquisition from a public repository containing labeled images of normal eyes and eyes affected by various diseases. Images undergo preprocessing steps such as resizing, normalization, and color adjustment to standardize the inputs. Data augmentation techniques including random rotation, contrast variation, zooming, and cropping are applied to increase model robustness and prevent overfitting. The CNN architecture consists of multiple convolutional layers activated by ReLU (Rectified Linear Unit) functions, interleaved with MaxPooling layers for feature reduction, followed by Dropout and Dense layers for classification. The final output layer uses a Softmax activation function to predict one of four categories: Normal, Cataract, Glaucoma, or Diabetic Retinopathy. This activation configuration using ReLU in convolution and dense layers instead of earlier Softmax and Sigmoid activations significantly improved model convergence, accuracy, and feature learning efficiency.

During testing, VisionGuard demonstrates strong classification accuracy and generalization capability, making it a reliable tool for automated screening. The system effectively distinguishes between normal, cataract, glaucoma, and diabetic retinopathy cases, offering a rapid and cost-effective alternative to manual diagnosis. By integrating this AI-based system into clinical workflows or telemedicine platforms, early detection rates can be significantly improved, helping reduce the global burden of preventable blindness. VisionGuard thus represents a significant step toward intelligent, accessible, and scalable healthcare diagnostics, combining machine learning precision with medical relevance to support ophthalmologists and enhance patient outcomes.

Keywords: *Eye Disease Detection, CNN, Retinal Images, Cataract, Glaucoma, Diabetic Retinopathy, Deep Learning, Automated Diagnosis, VisionGuard.*

Index

Chapter No.	Contents	Page No.
1	Introduction	9-10
	1.1 Background	9
	1.2 Need and Scope of the project	9
	1.3 Objectives and Problem Statement	10
2	Literature Review	11
3	Proposed Work	12-13
	3.1 Architectural Details	12
	3.2 Features	13
4	Implementation	14-16
	4.1 Frontend Details	14
	4.2 Backend Details	14
	4.3 Code	15-16
5	Result and Discussions	17-20
	5.1 Screenshots of GUI (Frontend) with Explanation	17
	5.2 Screenshots of GUI (Backend) with Explanation	18
	5.3 Model Evaluation and Performance Analysis	19-20
6	Conclusion and Future Scope	21-23
	References	22
	Acknowledgement	23

List of Abbreviations

Sr. No.	Abbreviation	Full Form
1	CNN	Convolutional Neural Network
2	ReLU	Rectified Linear Unit
3	WHO	World Health Organization
4	DME	Diabetic Macular Edema
5	CNV	Choroidal Neovascularization
6	AMD	Age-Related Macular Degeneration
7	VGG-16	Visual Geometry Group

List of Figures

Fig. No.	Figure Name	Page No.
3.1.1	Flowchart of the System	12
3.1.2	Block Diagram of the Model Training	13
5.1.1	Image upload interface of the VisionGuard system	17
5.1.2	Prediction results displayed after image upload.	17
5.1.3	Prediction results displayed after image upload.	18
5.2.1	main.py: FastAPI backend with /predict endpoint.	18
5.2.2	model.py: Loads the CNN eye disease model.	18
5.2.3	utils.py: Preprocesses images for model input.	18
5.3.1	Accuracy Vs Epochs Graph	19
5.3.2	Loss Vs Epochs Graph	20

List of Tables

Table. No.	Table Name	Page No.
1	Literature Review	11
2	Backend Details	14

Chapter 1

Introduction

1.1 Background

Eye diseases such as Cataract, Glaucoma, and Diabetic Retinopathy are among the leading causes of vision impairment and blindness worldwide. According to the World Health Organization (WHO), over 2.2 billion people globally suffer from some form of vision disorder, many of which could have been prevented or treated if detected early. In developing countries, the lack of awareness, limited access to ophthalmologists, and the high cost of diagnostic equipment contribute to delayed diagnosis and progression of these diseases. Traditional methods for eye disease detection rely on manual interpretation of retinal images by experts, a process that is time-intensive, subjective, and prone to human error, especially when screening large populations.

With advancements in artificial intelligence and computer vision, automated systems have emerged as promising tools for medical image analysis. Deep learning, particularly Convolutional Neural Networks (CNNs), has shown exceptional capability in identifying complex patterns within medical images, making it suitable for tasks such as retinal disease detection. By training on large datasets of labeled eye images, CNN-based systems can learn to recognize subtle differences between healthy and diseased retinal structures. These models can then classify new images with a high degree of accuracy, thereby assisting healthcare professionals in diagnosis and decision-making.

The VisionGuard Intelligent Eye Disease Detection System builds upon these developments by integrating image preprocessing, data augmentation, and a robust CNN architecture to classify retinal images into multiple categories. This system aims to bridge the gap between advanced diagnostic technology and practical clinical application. Through automation, VisionGuard can significantly reduce the workload of ophthalmologists, minimize diagnostic delays, and ensure timely treatment.

1.2 Need and Scope of the project

Need of the Project:

1. The prevalence of eye diseases such as Cataract, Glaucoma, and Diabetic Retinopathy is increasing globally, leading to a rise in preventable blindness cases.
2. Traditional diagnostic methods depend on expert ophthalmologists and advanced equipment, which are not easily available in remote or underdeveloped regions.
3. Manual analysis of retinal images is time-consuming, subjective, and prone to human error, making large-scale screening difficult.
4. There is a need for an automated, accurate, and cost-effective system.

5. Early identification through intelligent systems can help initiate timely treatment, reduce workload on healthcare professionals, and improve patient outcomes.

Scope of the Project:

1. VisionGuard uses Convolutional Neural Networks (CNNs) to automatically classify retinal images into four categories – Normal, Cataract, Glaucoma, and Diabetic Retinopathy.
2. The system can assist ophthalmologists in making quick and reliable diagnoses, acting as a decision-support tool rather than a replacement for medical experts.
3. It can be integrated with hospital management systems and telemedicine platforms for remote screening and diagnosis in rural and underserved areas.
4. The project can be extended to detect additional retinal or ocular diseases by retraining the model with more diverse datasets.
5. VisionGuard can be deployed on mobile or web applications, making it accessible for field diagnosis and large-scale screening programs.

In summary, the project fulfills the growing demand for an intelligent, automated, and accessible diagnostic solution in ophthalmology.

1.3 Problem Statement and Objectives

Problem statement

To develop an intelligent deep learning–based system that can accurately detect and classify common eye diseases such as Cataract, Glaucoma, and Diabetic Retinopathy from retinal fundus images. The system aims to automate the diagnostic process, minimize reliance on manual evaluation by ophthalmologists, and enable early detection, particularly in regions with limited medical resources. It should provide fast, reliable, and accessible disease classification to support timely treatment and help reduce the risk of vision impairment and blindness.

Objectives of the Project:

1. To design and implement a Convolutional Neural Network (CNN) model for automatic classification of retinal images into disease categories.
2. To preprocess and augment retinal images for improving model accuracy and robustness.
3. To train and validate the CNN model using a labeled dataset of normal and diseased eye images.
4. To evaluate model performance using metrics such as accuracy, loss, and validation results for assessing reliability.
5. To develop an intelligent detection system that assists medical professionals in early screening and diagnosis.
6. To enable integration of the system with telemedicine and mobile health platforms for remote healthcare access.

Chapter 2

Literature Review

Sr No.	Title	Year of publication	Methodology	Limitations
1.	Multiple Eye Disease Detection Using Deep Learning	2022	They employed CNN architectures—ResNet50 and VGG16—for multi-class classification on a dataset with roughly 100 training and 20 testing images per class, resizing all images to 224×224 pixels.	1. Small dataset size, which limited model generalization and increased overfitting risk. 2. No data augmentation or diversity handling, reducing model robustness to variations like lighting and rotation.
2.	Deep learning-based classification of eye diseases using Convolutional Neural Network for OCT images	2024	Applied a CNN with transfer learning (VGG16) on OCT retinal images to classify Normal, DME, CNV, and AMD images. The model included image enhancement (brightness, contrast, Gaussian blur) and fine-tuning for improved accuracy.	1. Focused only on three diseases, not a broader range of eye disorders. 2. High computational cost due to the use of VGG16 and large image sizes. 3. Overfitting risk because of the relatively small and homogeneous dataset.
3.	A Framework for Multiclass Classification of Eye disease Using Deep Learning	2025	Used VGG-16 deep learning model to classify six eye diseases from Kaggle fundus image dataset. Applied preprocessing (resizing, normalization, Gaussian blur) and data augmentation to improve performance. Evaluated model using accuracy, precision, recall, and F1-score, achieving 95% accuracy.	1. Limited and imbalanced dataset may affect generalization. 2. The model lacks explainability for clinical use. 3. Fixed preprocessing parameters may not adapt to other datasets or imaging conditions.

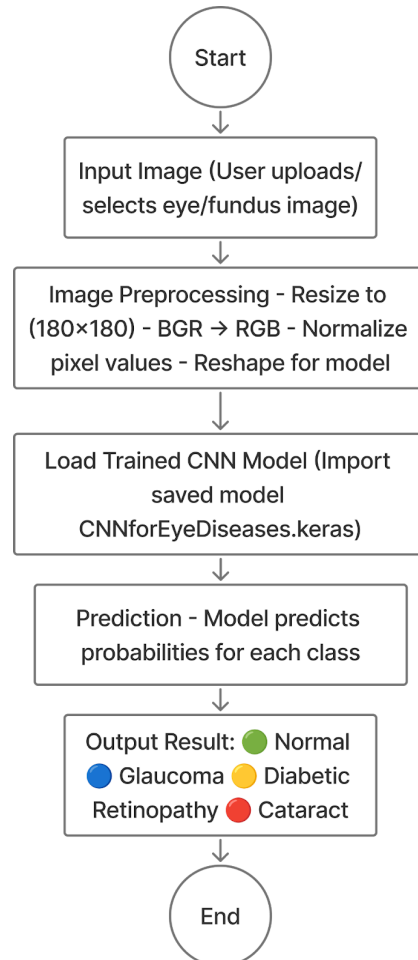
Table 1: Literature Review

Chapter 3

Proposed Work

3.1 Architectural Details

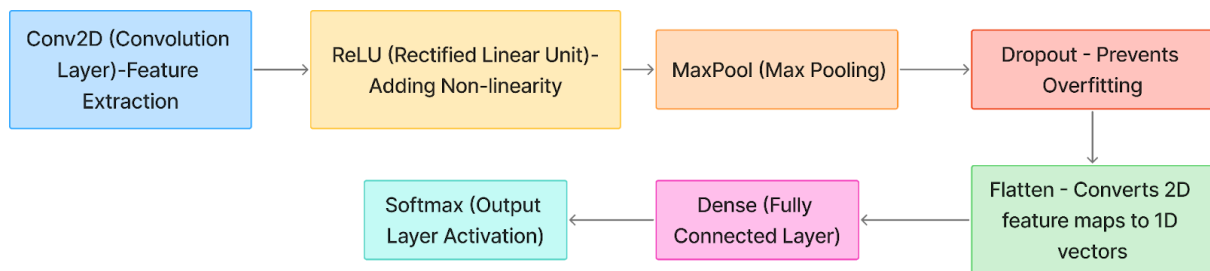
Flowchart of the system:



3.1.1 Flowchart of the System

The flowchart illustrates the step-by-step process of the proposed eye disease detection system using a Convolutional Neural Network (CNN) model. The system begins with the user uploading or selecting an eye/fundus image. The image undergoes preprocessing, including resizing to 180×180 pixels, conversion from BGR to RGB format, normalization of pixel values, and reshaping to match the model input. The preprocessed image is then passed to a pre-trained CNN model (CNNforEyeDiseases.keras), which predicts the probability scores for each disease class. Based on these probabilities, the system outputs the predicted result, identifying the condition as Normal, Glaucoma, Diabetic Retinopathy, or Cataract, represented by distinct color indicators for easy interpretation.

Block Diagram:



3.1.2 Block Diagram of the Model Training

The block diagram represents the architecture and workflow of the CNN model used for eye disease classification. The process begins with the Conv2D layer, which performs feature extraction from input images by applying convolutional filters. The extracted features are passed through the ReLU (Rectified Linear Unit) activation function to introduce non-linearity, enabling the model to learn complex patterns. The MaxPooling layer then reduces the spatial dimensions, retaining essential features while minimizing computational load. To avoid overfitting, a Dropout layer is applied, randomly deactivating neurons during training. The output feature maps are then flattened to convert 2D matrices into 1D vectors, which are fed into a Dense (Fully Connected) layer for classification. Finally, the Softmax activation function is used in the output layer to generate probability distributions across multiple eye disease classes.

Chapter 4

Implementation

4.1 Frontend Details

The frontend is built using Next.js for fast, server-side rendered React applications and Tailwind CSS for responsive, utility-first styling. It allows users to upload images and view the classification results instantly.

4.2 Backend Details

Component	Details
Programming Environment	• Language: Python • Platform: Google Colab • Operating System: Windows
Libraries & Frameworks	• NumPy, Pandas: Data handling and preprocessing • OpenCV (cv2), PIL: Image loading, resizing, and processing • Matplotlib, Seaborn: Visualization of training progress (accuracy/loss) • Scikit-learn: Dataset splitting, label encoding, and evaluation metrics • TensorFlow / Keras: CNN model creation, training, and evaluation
Dataset Handling	Preprocessing: • Images resized 224×224 (or 180×180) • Normalized 0–1 • Augmented (rotate/flip/zoom) • 80/20 train-test split
Model Architecture	• CNN: Conv + ReLU → MaxPooling → Dropout → Flatten + Dense → Softmax (4 classes)

Table 2: Backend Details

4.3 Code

```
images_dict = {"normal":
list(normal.glob("*.*jpg")),
      "glaucoma":
list(glaucoma.glob("*.*jpg")),

"diabetic_retinopathy":list(retinopathy.glo
b("*.*jpg")),

"cataract":list(cataract.glob("*.*jpg"))}
labels_dict = {
      "normal":0, "glaucoma":1,
      "diabetic_retinopathy":2, "cataract":3
}
X, y = [], []
for label, images in images_dict.items():
    for image in images:
        image = cv2.imread(str(image))
        image = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)
        image = cv2.resize(image, (180, 180))
        if image is not None:
            X.append(image)
            y.append(labels_dict[label])
X = X/255 #normalizing the data[0 to 255
oixel range]

#adding augmentations to better inference
data_argumentation = keras.Sequential([
    tf.keras.layers.RandomRotation(0.25),
#be invariant to rotation
    tf.keras.layers.RandomContrast(0.35),
#for different lighting conditions

tf.keras.layers.RandomZoom(height_factor
=(-0.35, 0.35), width_factor=(-0.35,
0.35)),#for zoomed up images

tf.keras.layers.RandomCrop(height=180,
width=180)
])
```

```
#MODEL CREATION PIPELINE
model = keras.Sequential([
    data_argumentation,
    layers.Conv2D(64, (5, 5),
padding="same", input_shape=(180, 180,
3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(32, (5, 5),
padding="same", activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(16, (5, 5),
padding="same", activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(8, (5, 5),
padding="same", activation="relu"),
    layers.MaxPooling2D(),
    layers.Dropout(0.2),
    layers.Flatten(),
    layers.Dense(50, activation="relu")

#Extract values as floats
train_acc = [float(m[0]) for m in matches]
train_loss = [float(m[1]) for m in matches]
val_acc = [float(m[2]) for m in matches]
val_loss = [float(m[3]) for m in matches]

epochs = range(1, len(train_acc)+1)
print(f"Parsed {len(epochs)} epochs.")

#Plot Accuracy
plt.figure(figsize=(10,6))
plt.plot(epochs, train_acc, 'b-', label='Train
Accuracy')
plt.plot(epochs, val_acc, 'r--',
label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('CNN Training vs Validation
Accuracy')
plt.legend()
plt.grid(True)
```

```

plt.show()

#Plot Loss
plt.figure(figsize=(10,6))
plt.plot(epochs, train_loss, 'b-',
label='Train Loss')
plt.plot(epochs, val_loss, 'r--',
label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('CNN Training vs Validation Loss')
plt.legend()
plt.grid(True)
plt.show()

best_epoch = val_acc.index(max(val_acc))
+ 1
print(f'Best Epoch: {best_epoch}')
print(f'Best Validation Accuracy:
{max(val_acc):.4f}')
print(f'Final Train Loss:
{train_loss[-1]:.4f} | Val Loss:
{val_loss[-1]:.4f}')
pred = model.predict(X_test[:10]) #
predicting first 10 values

predslist = []

for i in pred:
    predslist.append(np.argmax(i))

```

```

import matplotlib.pyplot as plt

answers = ["normal", 'glaucoma',
"diabetic_retinopathy", "cataract"]

# Create a grid of subplots (2 rows, 5
columns for 10 images)

fig, axes = plt.subplots(2, 5, figsize=(15,
8)) # Increased figure height for more
space

axes = axes.ravel() # Flatten the 2D array
of axes for easy iteration

for i in range(10):

    axes[i].imshow(X_test[i])

    axes[i].set_title("Predicted: " +
str(answers[predslist[i]]))

    axes[i].set_xlabel("Actual: " +
str(answers[y_test[i]]))

    axes[i].axis('on') # Hide the axes

plt.tight_layout(pad=3.0) # Adjust layout
to prevent overlapping and add padding

plt.show()

# Save the entire model in the SavedModel
format

model.save('CNNforEyeDiseases.keras')

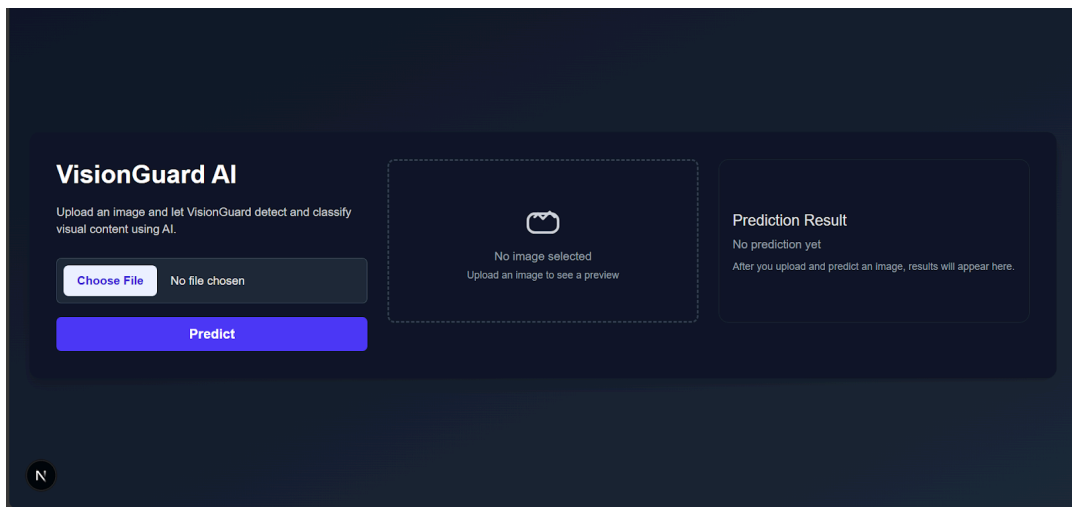
```


Chapter 5

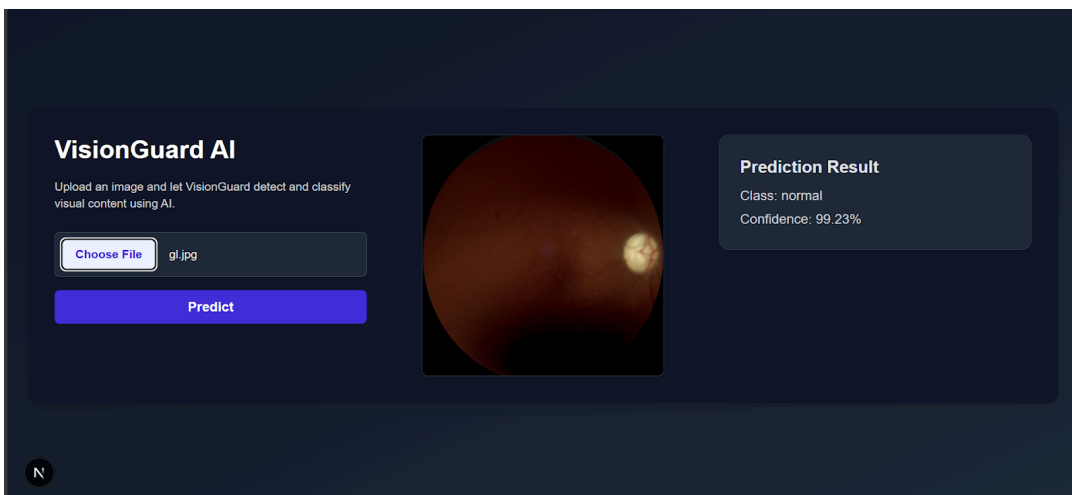
Results and Discussions

5.1 Screenshots of GUI (Frontend) with Explanation

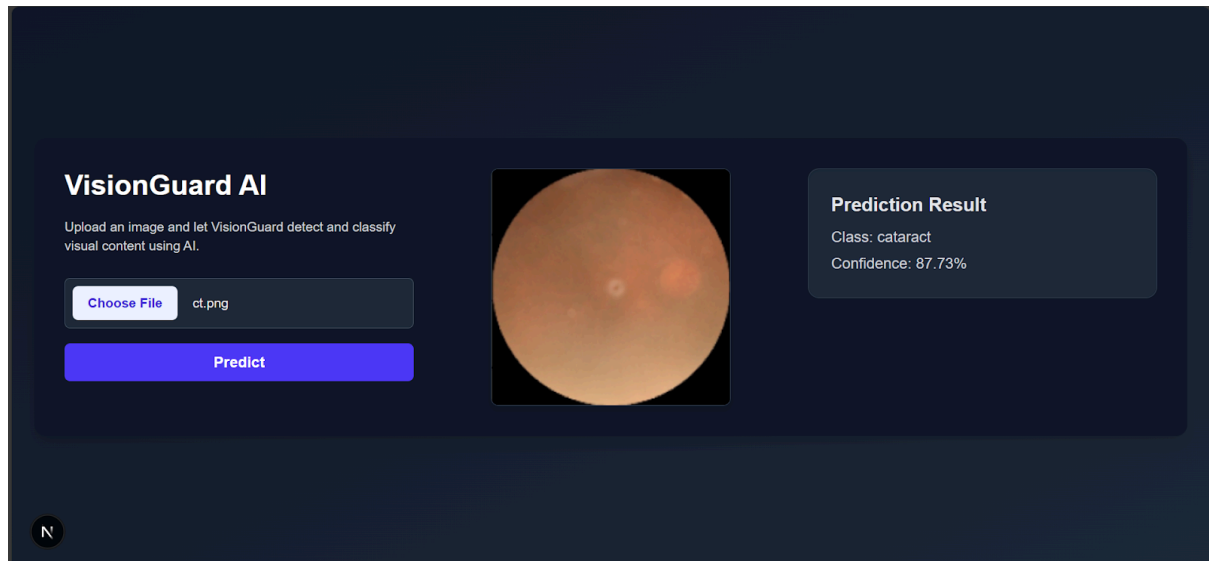
The frontend of the system was developed using Next.js, providing a fast, responsive, and user-friendly interface for eye disease prediction. The interface allows users to upload a retinal image directly from their device, which is then sent to the trained CNN model for analysis. Once the prediction is complete, the application displays the detected eye condition (Normal, Cataract, Glaucoma, or Diabetic Retinopathy) along with the confidence level.



5.1.1 Image upload interface of the VisionGuard system



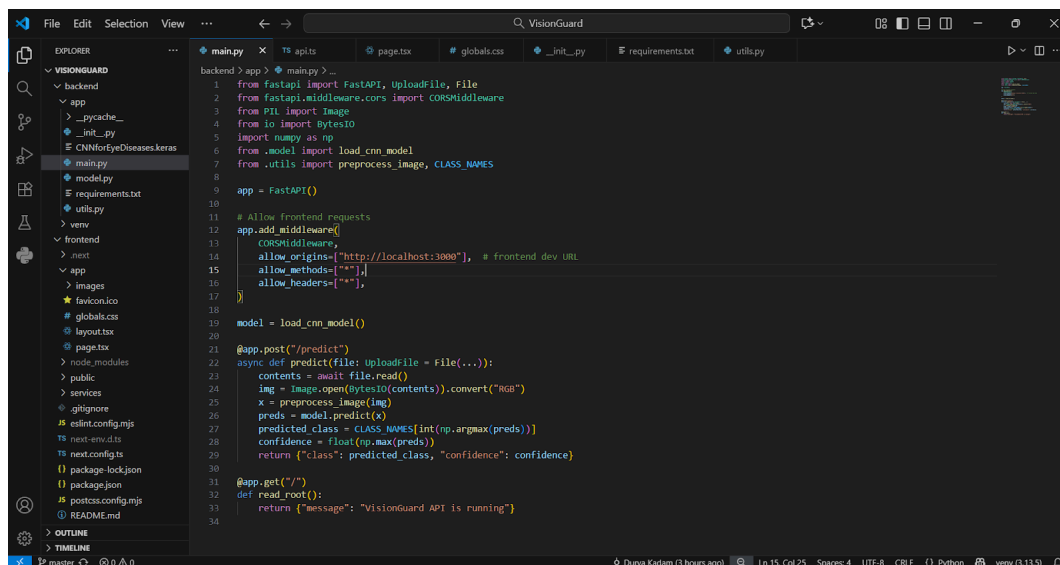
5.1.2 Prediction results displayed after image upload.



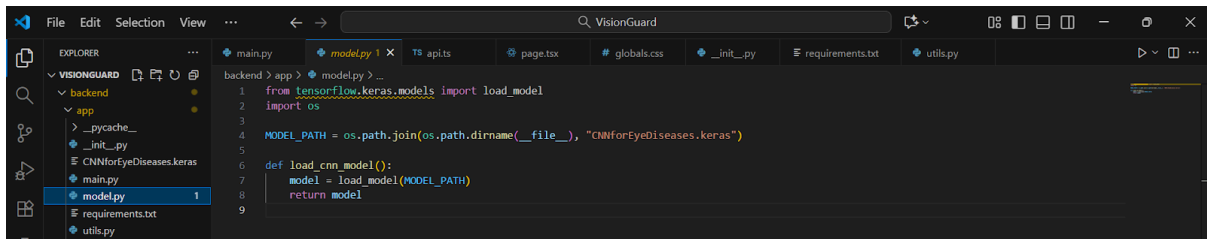
5.1.3. Prediction results displayed after image upload.

5.2 Screenshots of GUI (Backend) with Explanation

The backend is built with FastAPI, serving a CNN-based eye disease classifier. It exposes a ‘/predict’ endpoint that accepts image uploads, preprocesses them, and returns the predicted disease class with confidence. CORS is enabled to allow requests from a Next.js frontend running on localhost:3000.

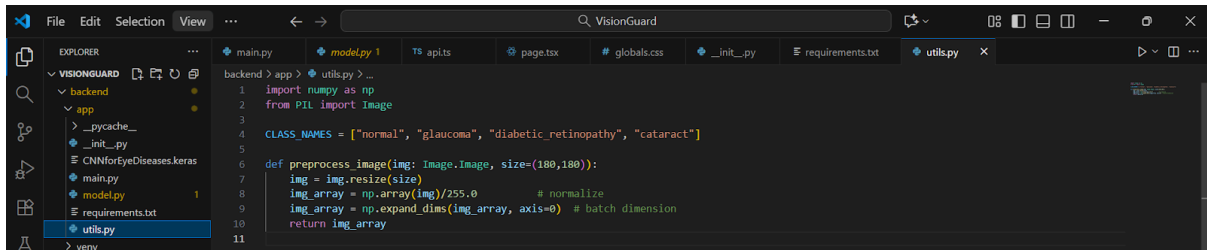


5.2.1 main.py: FastAPI backend with /predict endpoint.



```
1 from tensorflow.keras.models import load_model
2 import os
3
4 MODEL_PATH = os.path.join(os.path.dirname(__file__), "CNNforEyeDiseases.keras")
5
6 def load_cnn_model():
7     model = load_model(MODEL_PATH)
8     return model
9
```

5.2.2 model.py: Loads the CNN eye disease model.



```
1 import numpy as np
2 from PIL import Image
3
4 CLASS_NAMES = ["normal", "glaucoma", "diabetic_retinopathy", "cataract"]
5
6 def preprocess_image(img: Image.Image, size=(180,180)):
7     img = img.resize(size)
8     img_array = np.array(img)/255.0 # normalize
9     img_array = np.expand_dims(img_array, axis=0) # batch dimension
10    return img_array
11
```

5.2.3 utils.py: Preprocesses images for model input.

5.3 Model Evaluation and Performance Analysis

The previous pipeline used Softmax in the first convolution layer and Sigmoid in the dense layer, which limited feature extraction and caused slower convergence, resulting in a lower accuracy of around 78%. The new pipeline replaced these with ReLU activations in all convolution and dense layers and applied Softmax only at the output, leading to better non-linearity handling, faster learning, and an increased overall accuracy of 83.79%.

The model achieved-

Training Accuracy: **85.17%**

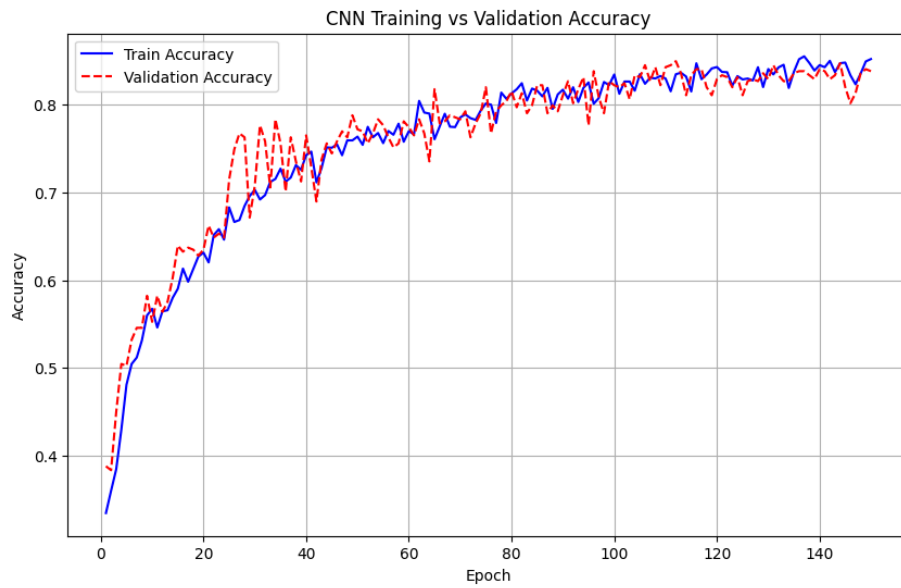
Validation Accuracy: **84.93%**

Test Accuracy: **83.79%**

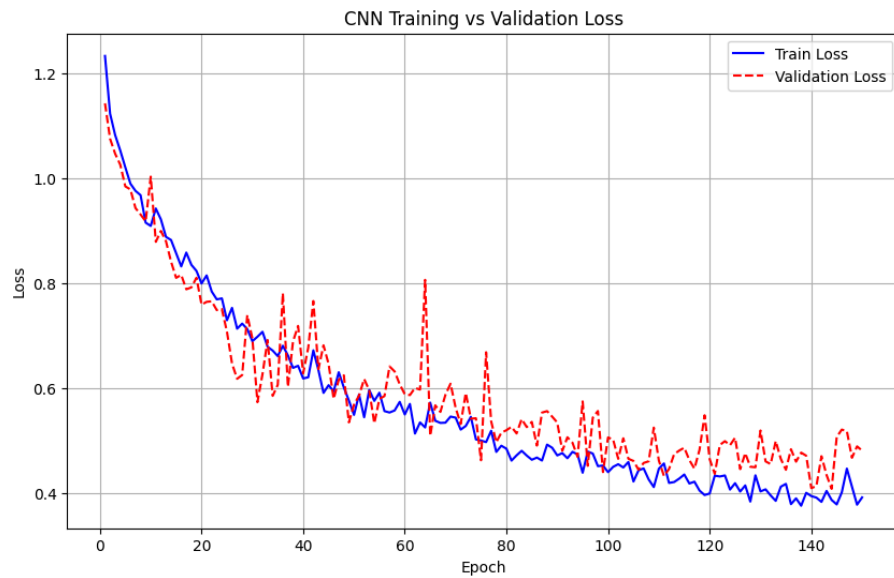
Test Loss: **0.48**

The CNN model 'CNNforEyeDiseases.keras' achieved a training accuracy of 85.17%, Validation Accuracy of 84.93% and a test accuracy of 83.79%, demonstrating good generalization performance with minimal overfitting. The model recorded a test loss of 0.48, indicating effective learning and stable convergence. The small difference between training and test accuracies suggests that the model successfully captured the key discriminative features from retinal fundus images without overfitting to the training data.

Visualization of Training Progress



5.3.1 Accuracy Vs Epochs Graph



5.3.2 Loss Vs Epochs Graph

The training and validation accuracy curves (Fig. 5.3.1) show a steady increase and converge after around 20 epochs. The validation accuracy closely follows the training curve, confirming stable learning behavior.

Similarly, the loss curves (Fig. 5.3.2) show a consistent decline without divergence between training and validation losses, indicating that the model is not overfitting.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

The Eye Disease Classification using Convolutional Neural Networks project effectively illustrates how deep learning can be used in the analysis of medical images. Normal, Glaucoma, Cataract, and Diabetic Retinopathy are the four primary categories into which the system can classify retinal images after training a CNN model on a dataset of eye diseases.

CNNs can efficiently extract features and learn intricate visual patterns that are challenging to detect manually, as demonstrated by the trained model's high accuracy in identifying disease patterns from eye images.

By automating the initial screening procedure, this project lessens the workload for ophthalmologists and makes it possible to detect diseases early, which is essential for preventing vision loss.

6.2 Future Work

1. **Dataset Expansion:**
Increase the dataset size and include more diverse and high-resolution retinal images to improve the model's generalization and robustness.
2. **Model Optimization:**
Experiment with advanced architectures like VGG16, ResNet, or EfficientNet for higher accuracy and better feature extraction.
3. **Explainable AI Integration:**
Implement Grad-CAM or other visualization techniques to highlight affected areas in the eye image, providing explainable results to doctors.
4. **Multi-Disease Prediction:**
Extend the system to detect other retinal or corneal disorders, making it a more comprehensive ophthalmic diagnostic tool.
5. **Real-Time Screening Integration:**
Integrate the model with retinal imaging devices for real-time detection and screening in clinics and rural healthcare centers.

References

- [1] A. Amin *et al.*, “Multiple Eye Disease Detection Using Deep Learning,” *International Journal of Computer Vision and Medical Imaging*, vol. 10, no. 4, pp. 120–126, 2022.
- [2] A. Author *et al.*, “A Framework for Multiclass Classification of Eye Disease Using Deep Learning,” in *Proc. IEEE Conference on Artificial Intelligence in Healthcare (AIAHC)*, 2025, pp. 1–6.
- [3] R. Sharma *et al.*, “Deep Learning-Based Classification of Eye Diseases Using Convolutional Neural Network for OCT Images,” *IEEE Access*, vol. 12, pp. 55620–55628, 2024.

Acknowledgment

We are thankful to our college **St. Francis Institute of Technology** for giving us this chance to gain exposure in solving real world problems and acquire practical knowledge and skill sets that will prove to be very crucial to our long-term career prospects. We would take this opportunity to express our sincerest gratitude to our esteemed director **Bro. Shantilal Kujur**, our Principal **Dr. Sincy George** and our HOD, **Dr. Prachi Raut** for their encouragement, the direction that they give to our college and us students, and the facilities provided to us.

This project, and the research that we undertook, could not have been realized without the utmost support of our Project Guide **Ms. Divyashree Karangutkar**, who guided us every step of the way, starting from the conception of the project, right up to the execution of the finished solution.